Learning to Select Context per Code Diff: An Accuracy–Cost Trade-off for LLM Code Review

Michael Liu (msl196) **Ruiding Feng (rf711)** **Li-Hua Peng (lp880)**
Machine Learning II, Rutgers University

Abstract

We study whether selecting context per code change can reduce the input cost of LLM-assisted review while maintaining accuracy. Our planned method trains a small selector for a frozen reviewer. We prepared a repository-based split of 46,518 ContextCRBench examples, completed a Qwen3B QLoRA pilot, and implemented context-policy baselines. The saved pilot outputs reveal malformed base-model answers and an adapter that always predicts the majority class. These findings motivate preprocessing and evaluation changes; they do not establish improved review quality.

Repository: `https://github.com/michaelsliu3/CS536_Project`

1 Research question

Reviewing a change may require its issue description or nearby code, but including every available field increases input cost. We ask: *Can a learned selector choose context per diff while maintaining review accuracy and using fewer tokens than fixed context choices?*

ContextCRBench provides issue, pull request (PR), and surrounding-code context [1]. Repoformer studies selective retrieval for code completion [2]; we investigate context selection for review, focusing on the accuracy–cost trade-off following instructor feedback.

We target accuracy within one percentage point of the best fixed strategy with at least 30% fewer extra context tokens, or higher accuracy at a matched budget. Our Qwen3B pilot supports RQ2 by testing whether a small adapted model can predict merge status when supplied all available context fields, subject to truncation. This establishes a baseline and exposes prerequisites for learning a selector; the selector's accuracy–cost benefit remains to be tested.

2 Data and experimental setup

Task and split. The reviewer returns a JSON answer with the Boolean field `is_merged`. Merge status is our benchmark target, but only a proxy for code quality: a merged change can contain a bug, and a rejection can have nontechnical reasons. We exclude explicit outcome fields and human review answers from prompts. Later edits and quoted solutions in issue and PR text still need a chronology audit.

Of 68,938 downloaded JSON records, cleaning removes 22,375 exact normalized-diff duplicates and 45 records from 16 conflicting-label groups, leaving 46,518 examples from 99 repositories. Seed 42 fixes an approximately 80/10/10 repository split, keeping PRs together. Repository sets are disjoint (Table 1); unequal repository sizes explain the example proportions. Validation supports development, while test repositories are reserved. Near-duplicates and temporal leakage remain to be checked.

Table 1: Retained data in the saved split. True and False denote merge status.

Split	Examples	Repositories	True	False	True (%)
Train	38,097	79	30,281	7,816	79.5
Validation	3,797	10	3,264	533	86.0
Test	4,624	10	3,336	1,288	72.1

Go (23.7%) and Python (17.4%) are the largest training languages; C# (39.4%) and Rust (27.8%) dominate validation.

A Python example. Training record `63859.json` comes from pandas PR #53773. The issue concerns a `TypeError` on nested JSON containing `null`. Below are four verbatim diff lines; others are omitted. The + marks additions.

```
+         if data is None:
+             return
          if isinstance(data, dict):
              data = [data]
```

The guard returns on `None`. Its label, `False`, records merge status, not fix correctness. This illustrates an input, not a prediction; the label stays out of the prompt.

Model and adaptation method. The pilot uses Qwen2.5-Coder-3B-Instruct, a decoder-only Transformer with 36 layers, hidden size 2,048, and feed-forward size 11,008. Grouped-query attention has 16 query heads and two key/value heads. It generates JSON autoregressively. Its configuration supports 32,768 positions; our experimental cap is 2,048 tokens.

QLoRA [3] freezes the base weights in 4-bit NF4 with double quantization and bfloat16 computation. We train low-rank adapters on the attention query, key, value, and output projections and the feed-forward gate, up, and down projections. Rank 16, scaling $\alpha = 32$, and dropout 0.05 give 29,933,568 trainable parameters. The saved run uses paged 8-bit AdamW, learning rate 2×10^{-4}, cosine decay, 3% warmup, and weight decay 0.01. Batch size one with eight accumulation steps gives an effective batch size of eight. Training ran for three epochs (14,289 optimizer steps) on one 32-GB RTX 5090; the final adapter is `checkpoint-14289`.

Inputs contain repository/language metadata, the diff, issue and PR text, and code before and after the change. Only JSON-answer and end-of-sequence tokens receive loss; prompt tokens are masked. After right truncation, 63.3% of training and 57.2% of validation sequences reach 2,048 tokens. These post-truncation lengths do not measure discarded tokens.

Evaluation and context policies. Evaluation compares the untouched base, the trained adapter, and a constant prediction chosen from training-label counts on the same saved 200 examples. The two model conditions share prompts, quantization, and deterministic decoding, with at most 2,016 input tokens and 32 generated tokens. Invalid JSON counts as an error and is reported separately. We report accuracy, macro-F1, and per-class recall alongside paired outputs.

Implemented policies include diff-only, all-context, fixed-mask, random, and BM25 similarity. They choose issue text, PR text, and before/after code, always retaining the diff and checking assembled prompt budgets. The planned compact Transformer selector will learn from training-repository reviewer outcomes. Context-policy evaluations and selector training remain pending; validation will determine fixed masks and budgets.

3 Progress and Preliminary Findings

We completed three epochs of QLoRA training with recoverable checkpoints and obtained paired predictions for the same 200 validation IDs. We checked protocol and source hashes and recomputed Table 2. The main challenge is interpreting the outputs: malformed base answers and majority-only adapter predictions prevent a claim of improved review quality. A corrected comparison remains pending.

Table 2: Saved validation pilot under the original strict-JSON protocol ($n = 200$). These diagnostics do not establish improved review quality.

Method	Accuracy (%)	Macro-F1	Invalid (%)
Training-majority constant	87.5	0.467	0.0
Untouched Qwen3B	0.0	0.000	100.0
Qwen3B + trained adapter	87.5	0.467	0.0

4 Lessons Learned and Limitations

Scope of the pilot. The adaptation pilot is preparatory evidence for RQ2: it asks whether fine-tuning a small model improves merge-status prediction with all available context fields supplied before truncation. It exercises training and paired evaluation and exposes preprocessing failures that would also compromise selector supervision. This is useful baseline work, but it does not yet train a context selector or measure savings from selecting less context. Those require holding the reviewer fixed while comparing learned subsets with the best fixed configuration.

Class imbalance. Training is 79.5% True; the evaluation sample has 175 True and 25 False examples. The adapter predicts True throughout, matching the majority baseline at 87.5% accuracy, macro-F1 0.467, and zero False recall. Accuracy alone therefore hides complete failure on the minority class. Substantial preprocessing and training-data auditing are needed before further modeling: we will inspect label provenance and test balanced training sampling or example-weighted answer loss using training-label frequencies. Imbalance is a measured problem, although its causal contribution to the collapse is not isolated. Validation macro-F1 and minority recall will guide selection; validation/test distributions will stay unchanged.

Output format and truncation. All 200 base answers fail strict parsing: 82 untruncated outputs contain Markdown fences, and CPU tokenization confirms that 118 truncated inputs lose the assistant-start suffix. The zero score reflects malformed evaluation inputs and nonconforming outputs, not evidence that Qwen cannot generate JSON or judge changes. The audit identifies chat construction and output-format failures; it does not establish that the underlying dataset JSON is malformed. We will budget context before rendering chats, preserve role boundaries and output space, and rerun both models under a new protocol. Constrained verdict decoding is another candidate. Original strict scores will be retained.

A verdict label is not a selector objective. The single `is_merged` label supplies a task target, but neither identifies useful context subsets nor measures their cost. RQ2 needs reviewer outcomes for different subsets of the same example, paired with actual input-token counts and selector overhead. The goal is to preserve predictive quality while supplying less *extra context*, retaining the diff. We will report accuracy, macro-F1 and minority recall against both total input tokens and extra context tokens; merge status alone cannot establish general review quality. Chronology and near-duplicate checks remain unfinished, and repository language differences limit generalization.

Learning, label quality, and stability. Beyond the 200-example result, within-repository AUROC is only 0.520 on validation and 0.505 on sampled training data, so the adapter did not learn to judge changes. Six training repositories with under 2% True labels hold 20.5% of False examples, suggesting label noise beyond the proxy issue; we will handle them before balancing to 1:1. Gradient norms were also unstable, peaking at 89,273, although validation had plateaued by step 1,000.

5 Next steps

First, audit labels and imbalance, repair context budgeting and chat formatting, and repeat the base-versus-adapter baseline under a new protocol. Keep the original outputs as failure diagnostics. Compare any more complex predictors under the same corrected split and protocol rather than treating model size as a remedy.

We will also host a model locally in LM Studio and use Alibaba's Open Code Review CLI (`ocr`) for an end-to-end terminal demonstration on validation examples. The current adapter cannot drive this demonstration: in a smoke test it returned only verdict JSON and made no tool calls, so the review failed, while a general tool-capable local model completed the same review. The demonstration will therefore use a tool-capable model until a suitable reviewer is ready. It will show the workflow, not establish review quality.

Next, freeze a reviewer and compare diff-only, issue, PR, surrounding-code and all-context policies on identical validation examples (RQ1). If outcomes vary usefully across subsets, generate subset-quality and token-cost supervision from training repositories and fine-tune the small selector (RQ2). Compare its accuracy–cost frontier with the best fixed policy, then analyze helpful, neutral and harmful context choices and selector failures (RQ3). Include repeated seeds, selection overhead and negative findings.

References

[1] R. Hu et al. Benchmarking LLMs for fine-grained code review with enriched context in practice. *arXiv:2511.07017*, 2025. `https://arxiv.org/abs/2511.07017`.

[2] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma. Repoformer: Selective retrieval for repository-level code completion. *ICML*, PMLR 235:53270–53290, 2024.

[3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv:2305.14314*, 2023.